Tema 6. Seguridad en aplicaciones

Añadiendo seguridad a nuestro proyecto

Índice

- Motivación
- SpringSecurity
  - Incorporando Spring Security
  - Usando Preconditions

Spring Security

- Para poder utilizar Sprint Security añadiremos la dependencia Maven necesaria

```
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

- Una vez añadida, al arrancar el proyecto SpringBoot generará un password automático de prueba para el usuario "user"

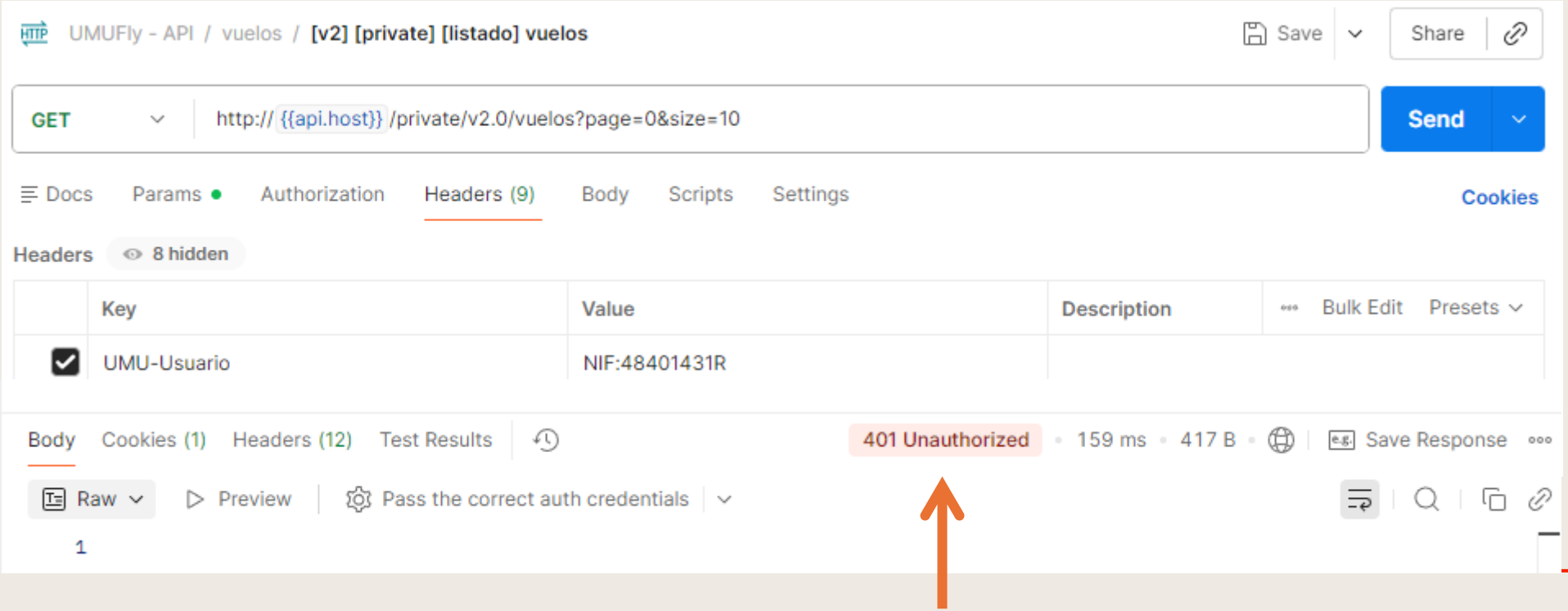
```
2026-03-07T21:01:31.435+01:00  WARN 17056 --- [umufly-api] [  restartedMain] .s.a.UserDetailsServiceAutoConfiguration :

Using generated security password: 7a0b36d5-37f1-4fcf-ae58-9129953321d0

This generated password is for development use only. Your security configuration must be updated before running your application in production.
```

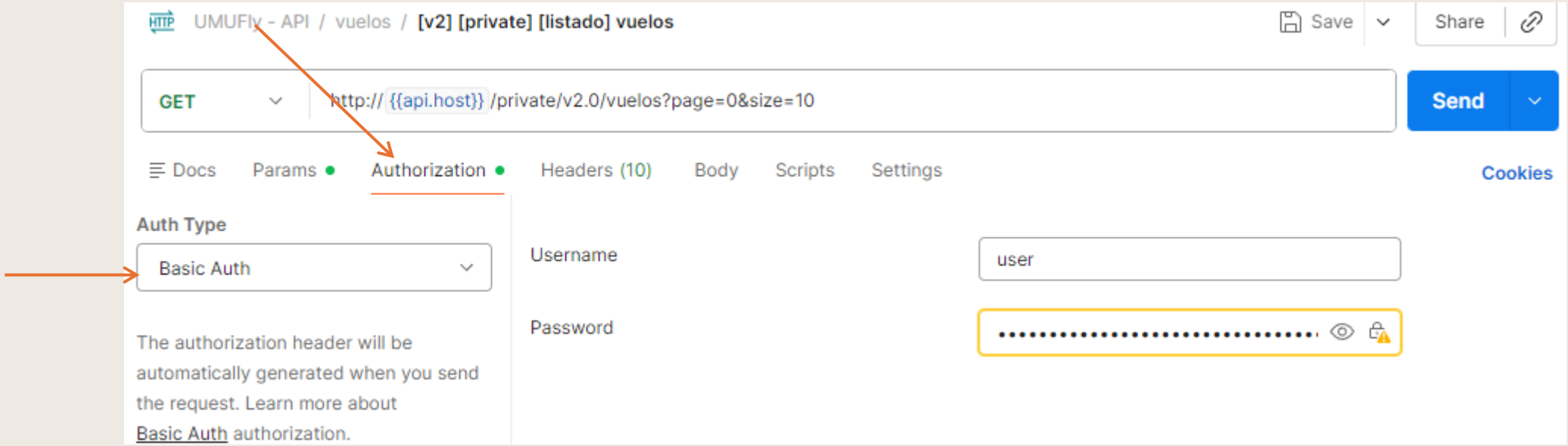
Spring Security

- Sólo con añadir la dependencia nuestra API ya requiere un flujo de autenticación.
- Si probamos nuestras peticiones guardadas en Postman



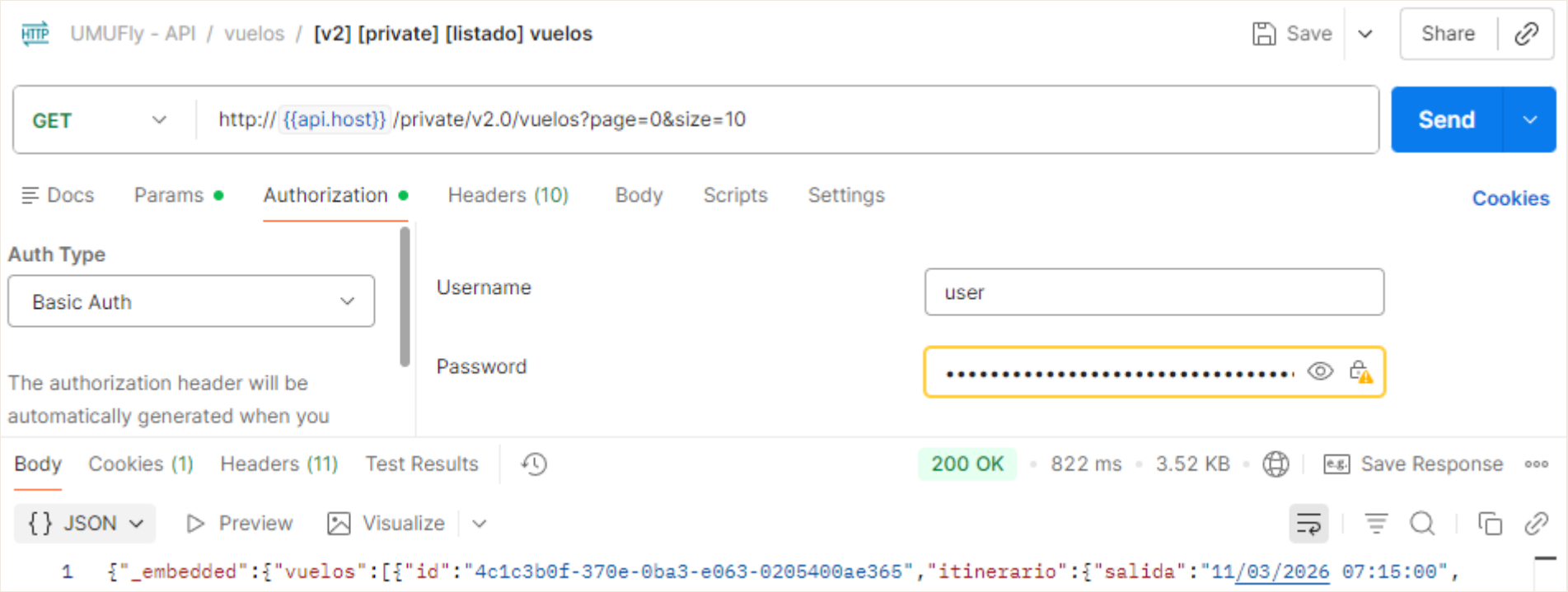
Spring Security

- SpringBoot nos pide una cabecera de autorización
- Introduzcamos la que nos indica al arrancar y veamos si podemos acceder a los métodos



Spring Security

- Ahora, SpringBoot autentica al usuario y permite ejecutar el endpoint



Spring Security

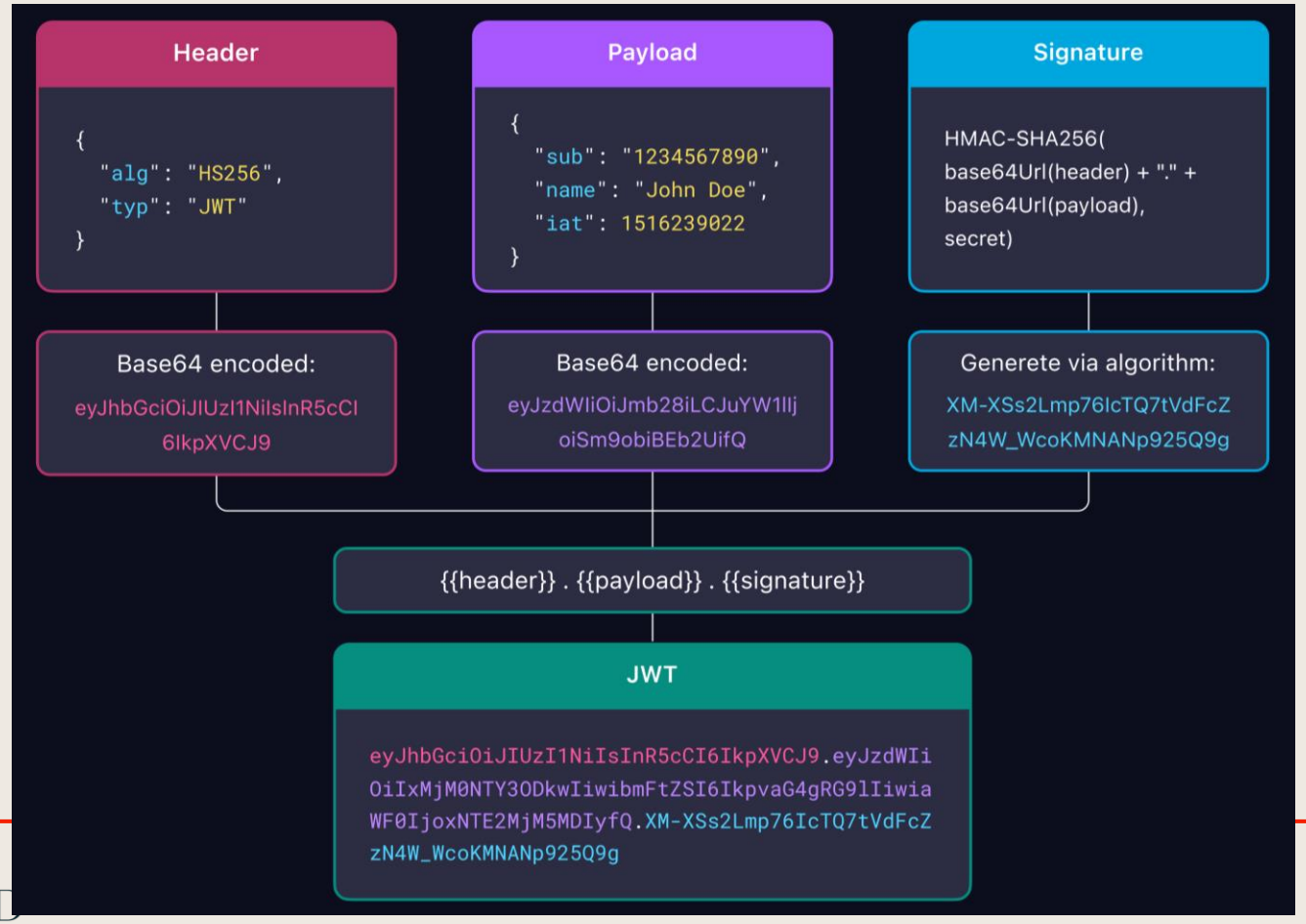
- Una vez tenemos autenticación debemos ajustar el mecanismo
- No queremos usar autenticación básica
- Los usuarios deben buscarse en nuestra base de datos
  - Nota: Se ha creado en el esquema una tabla para usuarios con:
    - Email
    - Password
    - Nombre

Spring Security

- Vamos a optar por intercambiar Json Web Tokens JWT:
  - Estándar abierto para transmitir información de forma segura
  - Compacto: se envía como cabecera HTTP
  - Seguro: la información va firmada por lo que se detectarán modificaciones
  - Autocontenido: puede incluir información del usuario para ahorrar buscarla en base de datos

Spring Security

- Formado por: header, payload y signature



Spring Security

- Vamos a optar por intercambiar Json Web Tokens JWT:
  - Algunos claims estándar:
    - sub: Subject, identifica al usuario
    - iss: Issuer, quién emite el token
    - aud: Audience, para quién está destinado el token
    - exp: Expiration, cuando expirará el token
    - jti: JWT ID, identificador único del token
    - iat: Issued at, cuándo se emitió el token

Spring Security

- Para poder usar JWT debemos añadir las dependencias necesarias

```
<dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-api</artifactId>
        <version>0.12.6</version>
</dependency>
<dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-impl</artifactId>
        <version>0.12.6</version>
        <scope>runtime</scope> <!- Imprescindible -->
</dependency>
<dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-jackson</artifactId>
        <version>0.12.6</version>
        <scope>runtime</scope> <!- Imprescindible -->
</dependency>
```

Spring Security

- Una vez tenemos todas las dependencias, creamos la Entidad que leerá de base de datos el usuario

```
@Entity
@Table(name = "USUARIO")
public class UsuarioEntity {

  private String email;
  private String password;
  private String nombre;

  @Id
  @Column(name = "EMAIL")
  public String getEmail() {
         return email;
  }

  public void setEmail(String email) {
         this.email = email;
  }
```

Spring Security

- Pero la entidad Usuario no es suficiente para SpringBoot
- Esta clase debe implementar UserDetails
- Esta interfaz de SpringBoot indica al sistema de autenticación que será la clase que gestione al usuario
- Deberemos implementar los métodos que nos pide:
  - UserDetails.getAuthorities: Devuelve los roles del usuario en el sistema
  - UserDetails.getUsername: Devuelve el identificador de usuario que se usa para autenticarlo

Spring Security

- Los nuevos métodos debemos anotarlos con @Transient para que hibernate no busque las columnas

```
//Creamos temporalmente un rol administrador (no lo cargamos de base de datos)
//El rol debe empezar siempre por "ROLE_"
@Override
//Transient para que hibernate no lo considere una columna
@Transient
public Collection<? extends GrantedAuthority> getAuthorities() {
  return List.of(new SimpleGrantedAuthority("ROLE_ADMIN"));
}


//El identificador de usuario en el sistema sera el email
@Override
//Transient para que hibernate no lo considere una columna
@Transient
public String getUsername() {
  return getEmail();
}
```

Spring Security

- Con la entidad Usuario terminada, creamos el repositorio JPA para cargar los registros

```
@Repository
public interface JpaUsuarioRepository extends JpaRepository<Usuario, String> {

  Optional<Usuario> findByEmail(String email);

}
```

Spring Security

- Con el acceso a la base de datos creamos el servicio de generación y validación de tokens JWT

```java
@Service
public class JwtService {

@Value("${jwt.secret}")
private String secret;

@Value("${jwt.expiration-ms}")
private long expirationMs;

private SecretKey getKey() {
  return Keys.hmacShaKeyFor(Decoders.BASE64
        .decode(secret));
}
```

```java
//Generamos el token a partir del usuario
public String generarToken(UsuarioEntity usuario) {
return Jwts.builder()
  .subject(usuario.getUsername()) //Establecemos sub
  .claim("rol", usuario.getAuthorities()) //Claim extra: rol
  .claim("nombre", usuario.getNombre()) //Claim extra:nombre
  .issuedAt(new Date())
  .expiration(new Date(System.currentTimeMillis() + expirationMs))
  .signWith(getKey())
  .compact();
}
```

Spring Security

- Con el acceso a la base de datos creamos el servicio de generación y validación de tokens JWT

```
public Claims validarYExtraer(String token) {
  return Jwts.parser()
    .verifyWith(getKey())
    .build()
    .parseSignedClaims(token)
    .getPayload();
}


public String extraerUsername(String token) {
  return validarYExtraer(token).getSubject();
}
```

Spring Security

- Una vez podemos generar y validar necesitamos crear la clase encargada de obtener el usuario
- SpringBoot provee una interfaz para ello UserDetailsService
- A partir de un nombre de usuario debe ser capaz de devolver un objeto de tipo UserDetails
  - La clase Usuario que creamos implementa la interfaz UserDetails

Spring Security

- Una vez podemos generar y validar necesitamos crear la clase encargada de obtener el usuario
- SpringBoot provee una interfaz para ello UserDetailsService
- A partir de un nombre de usuario debe ser capaz de devolver un objeto de tipo UserDetails
  - La clase Usuario que creamos implementa la interfaz UserDetails
- Gracias a esta API podemos conectar automáticamente el sistema de autenticación y autorización con nuestra aplicación

Spring Security

- Una vez podemos generar y validar necesitamos crear la clase encargada de obtener el usuario

```
@Service
public class UsuarioDetailsService implements UserDetailsService {
        private final JpaUsuarioRepository usuarioRepository;

        public UsuarioDetailsService(JpaUsuarioRepository usuarioRepository) {
                this.usuarioRepository = usuarioRepository;
        }

        @Override
        public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
                // username tiene formato email@dominio.com
                // En nuestra bbdd: prueba@um.es
                return usuarioRepository.findByEmail(username)
                .orElseThrow(() -> new UsernameNotFoundException("Usuario no encontrado: " + username));
        }
}
```

Spring Security

- Finalmente debemos crear un interceptor que analice todas las peticiones:
  - Interceptaremos todas las peticiones que entran a la aplicación
  - Sólo lo haremos una vez por cada petición
  - Validaremos el token dentro del interceptor
    - Al ser un token firmado tenemos garantía que es el original
    - Al tener tiempo de expiración, sabemos si está vigente
    - SpringBoot lo validará de manera automática sin que tengamos que implementar nada
  - Recuperaremos el usuario que indica el token

Spring Security

- Finalmente debemos crear un interceptor que analice todas las peticiones:

Solo una vez por petición

Servicio validación del token

Servicio de carga de usuarios

```
@Component
public class JwtAuthFilter extends OncePerRequestFilter {

private final JwtService jwtService;
private final UsuarioDetailsService usuarioDetailsService;

 public JwtAuthFilter(JwtService jwtService, UsuarioDetailsService usuarioDetailsService) {
        this.jwtService = jwtService;
        this.usuarioDetailsService = usuarioDetailsService;
 }
}
```

Spring Security

- Ahora vamos a implementar el filtro

```
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain
filterChain) throws ServletException, IOException, java.io.IOException {
        //Obtengo la cabecera de autorizacion
        String authHeader = request.getHeader("Authorization");
        //Al ser JWT debe empezar por "Bearer "
        //Si no existe o no empieza con "Bearer " ojo va con un espacio en blanco
        //Continuo el filtrado (no autorizo la peticion)
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
                filterChain.doFilter(request, response);
                return;
        }

        …
        …
```

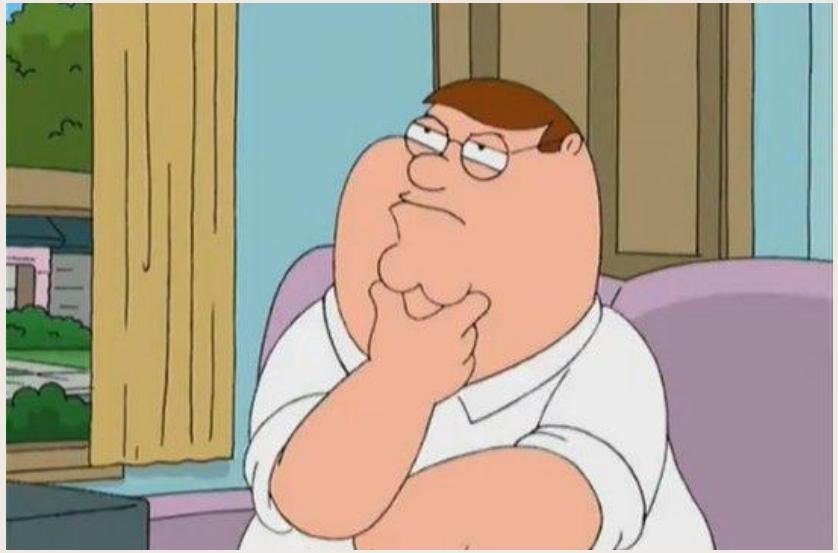
Spring Security

- Ahora vamos a implementar el filtro

```
try {
        //Extraigo el nombre de usuario
        String username = jwtService.extraerUsername(token);
        if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
                //Cargo el usuario en el sistema
                UserDetails userDetails = usuarioDetailsService.loadUserByUsername(username);
                //Creo un "principal" de autenticacion basdo en el flujo usuario-pasword
                //Cargo el usuario y sus roles, no incluyo credenciales ya que no las necesito (el campo null)
                UsernamePasswordAuthenticationToken auth = new UsernamePasswordAuthenticationToken(userDetails, null,
                        userDetails.getAuthorities());
                //Construyo un objeto WebAuthenticationDetails a partir de la peticion (guarda IP e identificador de sesion si hay)
                //Lo aniado a la credencial de autenticacion
                auth.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
                //Aniado al contexto de seguridad el "pricipal" de autenticacion relleno para que autorice la peticion
                SecurityContextHolder.getContext().setAuthentication(auth);
        }
    } catch (Exception e) {
        response.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Token inválido o expirado");
        return;
    }
filterChain.doFilter(request, response);
}
```

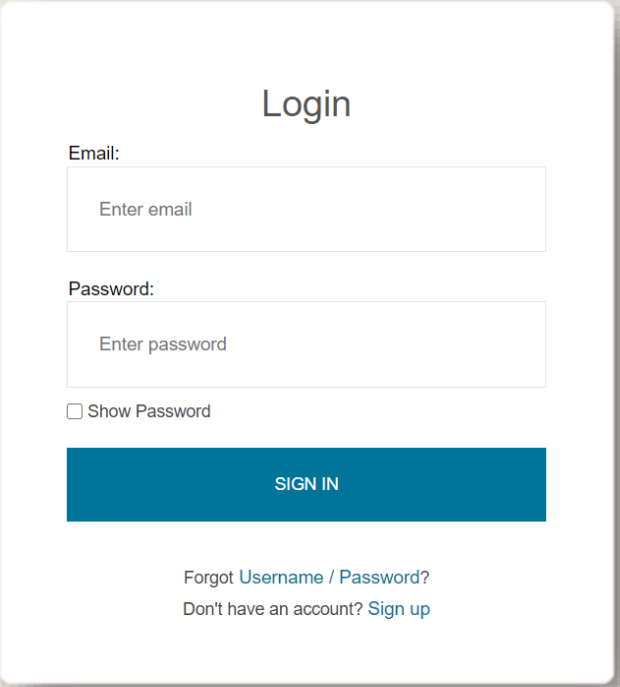
Spring Security

- En este punto ya tenemos el sistema de validación de token y recuperación del usuario
- Pero… ¿falta algo?



Spring Security

- En este punto ya tenemos el sistema de validación de token y recuperación del usuario
- Poder hacer login
- Poder bloquear las peticiones que requieran autenticación



Spring Security

- Debemos bloquear todos los accesos a la parte privada de nuestra API
- Debemos crear un Endpoint que nos permita enviar el usuario/password para obtener el token
- Ese endpoint debe ser público
- El resto de nuestra API debe estar dentro de una ruta privada

Spring Security

- El siguiente paso es crear un SecurityConfig que bloqueará los accesos a la parte privada de nuestra aplicación
- Obligará a que ese acceso sea autenticado
- Debermos crear también un encriptador de password, obligatorio para el flujo Usuario/Password en SpringBoot

Spring Security

- El siguiente paso es crear un SecurityConfig

Activa la gestión de la autenticación

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

        private final JwtAuthFilter jwtAuthFilter;

        public SecurityConfig(JwtAuthFilter jwtAuthFilter) {
                this.jwtAuthFilter = jwtAuthFilter;
        }
}
```

Spring Security

- El siguiente paso es crear un SecurityConfig:
  - Dentro crearemos los beans encargados de gestionar la seguridad

Carga un gestor de autenticación definido por la configuración establecida

```
@Bean
public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws
Exception {
        return config.getAuthenticationManager();
}
```

Carga un cifrador de password

```
@Bean
public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
}
```

Spring Security

- El siguiente paso es crear un SecurityConfig:
  - Dentro crearemos los beans encargados de gestionar la seguridad

Filtra las peticiones dejando sólo públicas las de login

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
  return http.csrf(csrf -> csrf.disable())
        .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(
                auth -> auth.requestMatchers("/auth/login").permitAll().anyRequest().authenticated())
        .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class).build();
}
```

Spring Security

- Por último, debemos habilitar el endpoint para poder hacer login
- El primer paso será crear dos records para la lectura y escritura de los datos

```
public record LoginRequestDTO(String email, String password) {}

public record TokenResponseDTO(String token) {}
```

Spring Security

- Habilitamos el endpoint para la autenticación

```
@RestController
@RequestMapping("/auth")
public class AuthController {

        private final JpaUsuarioRepository usuarioRepository;
        private final JwtService jwtService;
        private final PasswordEncoder passwordEncoder;

        public AuthController(JpaUsuarioRepository usuarioRepository, JwtService jwtService,
                PasswordEncoder passwordEncoder) {
                this.usuarioRepository = usuarioRepository;
                this.jwtService = jwtService;
                this.passwordEncoder = passwordEncoder;
        }
…
…
}
```

Spring Security

- Habilitamos el endpoint para la autenticación

```
@PostMapping("/login")
public ResponseEntity<TokenResponseDTO> login(@RequestBody LoginRequestDTO loginRequest) {

        UsuarioEntity usuario = usuarioRepository.findByEmail(loginRequest.email())
        .orElseThrow(() -> new ResponseStatusException(HttpStatus.UNAUTHORIZED, "Credenciales incorrectas"));


        if (!passwordEncoder.matches(loginRequest.password(), usuario.getPassword())) {
                throw new ResponseStatusException(HttpStatus.UNAUTHORIZED, "Credenciales incorrectas");
        }

        return ResponseEntity.ok(new TokenResponseDTO(jwtService.generarToken(usuario)));
}
```

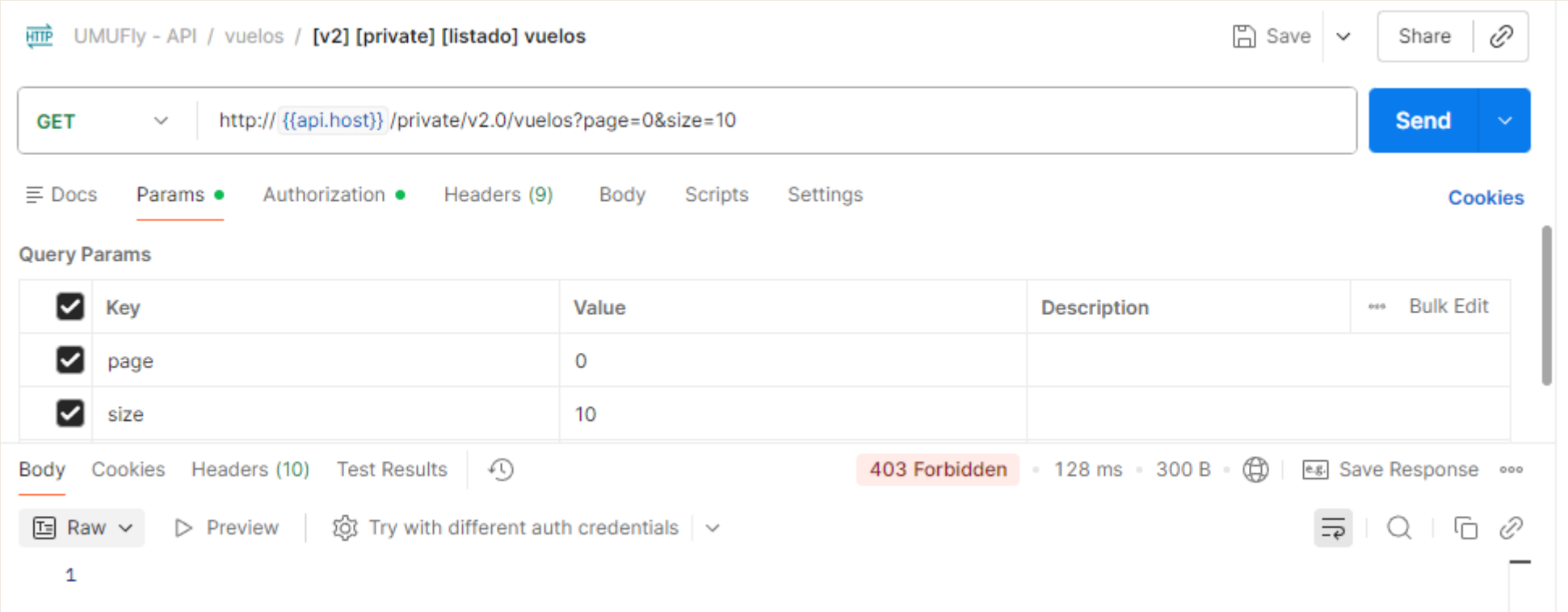
Spring Security

- Una vez terminada la configuración arrancamos el proyecto.
- SpringBoot detectará nuestro nuevo sistema indicándolo en el log
- Tampoco generará password aleatorio y el usuario "user"

```
i] [  restartedMain] o.s.d.j.r.query.QueryEnhancerFactories    : Hibernate is in classpath; If applicable, HQL parser will be used.
i] [  restartedMain] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view ren
i] [  restartedMain] r$InitializeUserDetailsManagerConfigurer : Global AuthenticationManager configured with UserDetailsService bean with name usuarioDetailsService
i] [  restartedMain] o.s.boot.tomcat.TomcatWebServer          : Tomcat started on port 8080 (http) with context path '/'
i] [  restartedMain] e.um.atica.umufly.UmuflyApiApplication   : Started UmuflyApiApplication in 21.017 seconds (process running for 21.758)
```

Spring Security

- Ahora si volvemos a llamar con Postman a cualquier endpoint dará error de antenticación



Spring Security

- Para poder acceder a los servicios deberemos pedir un token al sistema
  - Usaremos el endpoint que hemos creado
  - Para este ejemplo está creado el siguiente usuario:
    - Identificador: prueba@um.es
    - Password: 12345

Spring Security

- El servicio nos responderá con el token generado

UMUFly - API / **Login**

Save | Share

POST http://{{api.host}}/auth/login **Send**

Docs | Params | Authorization | Headers (9) | Body | Scripts | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Schema | Beautify

```
1 {
2     "email":"prueba@um.es",
3     "password": "12345"
4 }
```

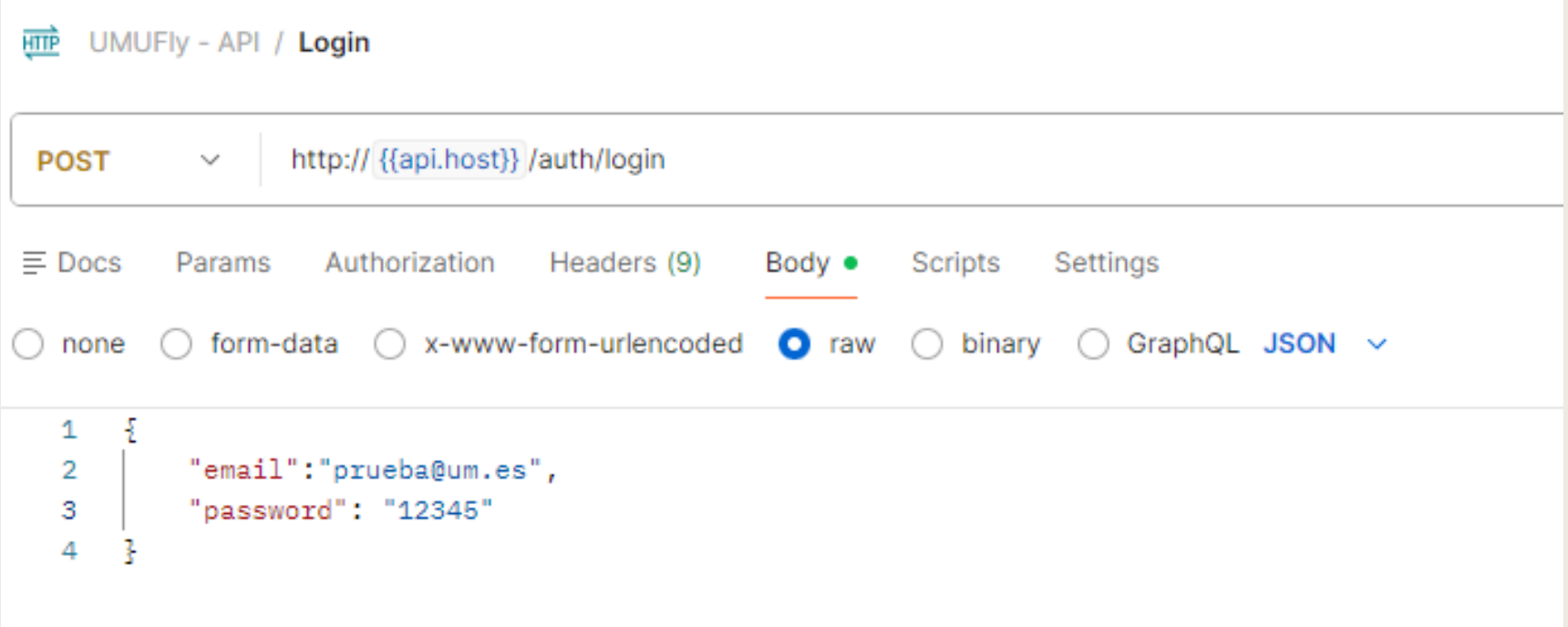
Body | Cookies | Headers (11) | Test Results | 200 OK | 535 ms | 581 B | Save Response

{} JSON | Preview | Visualize

```
1 {
2     "token": "eyJhbGciOiJIUzI1NiJ9.
        eyJzdWIiOiJwcnVlYmFAdW0uZXMiLCJyb2wiOlt7ImF1dGhvcml0eSI6IlJPTEVfQURNSU4ifV0sIm5vbWJyZSI6IlBydWViYSBNYXJ0aW5leiB
        DdWFkcmFkbyIsImlhdCI6MTc3MzAxMTk4NSwiZXhwIjoxNzczMDk4Mzg1fQ.d7blimkPPxkDuyE4B6y94KNaIK2VKvEv0jJ7Xs2exhs"
3 }
```

Spring Security

- En postman crearemos una petición POST al endpoint \auth\login
- Enviando un JSON con los campos email y password



Spring Security

- Si revisamos el token en https://www.jwt.io

ENCODED VALUE

Enable auto-focus

JSON WEB TOKEN (JWT) COPY CLEAR

Valid JWT

Invalid Signature

eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJwcnVlYmFAdW0uZXMiLCJyb2wiOlt7ImF1dGhvcml0eSI6IlJPTEVfQURNSU4ifV0sIm5vbWJyZSI6IlBydWViYSBNYXJ0aW5leiBDdWFkcmFkbyIsImlhdCI6MTc3MzAxMTk4NSwiZXhwIjoxNzczMDk4Mzg1fQ.d7blimkPPxkDuyE4B6y94KNaIK2VKvEv0jJ7Xs2exhs

DECODED HEADER

JSON CLAIMS TABLE COPY


```
{
  "alg": "HS256"
}
```


DECODED PAYLOAD

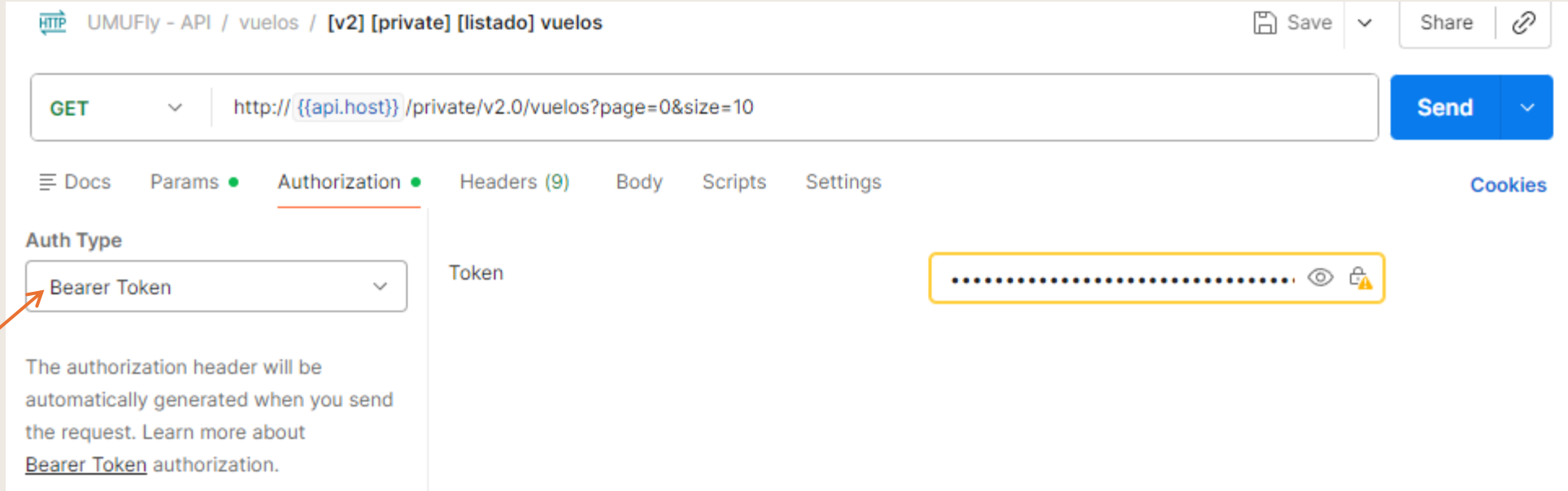
JSON CLAIMS TABLE COPY


```
{
  "sub": "prueba@um.es",
  "rol": [
    {
      "authority": "ROLE_ADMIN"
    }
  ],
  "nombre": "Prueba Martinez Cuadrado",
  "iat": 1773011985,
  "exp": 1773098385
}
```


JWT SIGNATURE VERIFICATION (OPTIONAL)

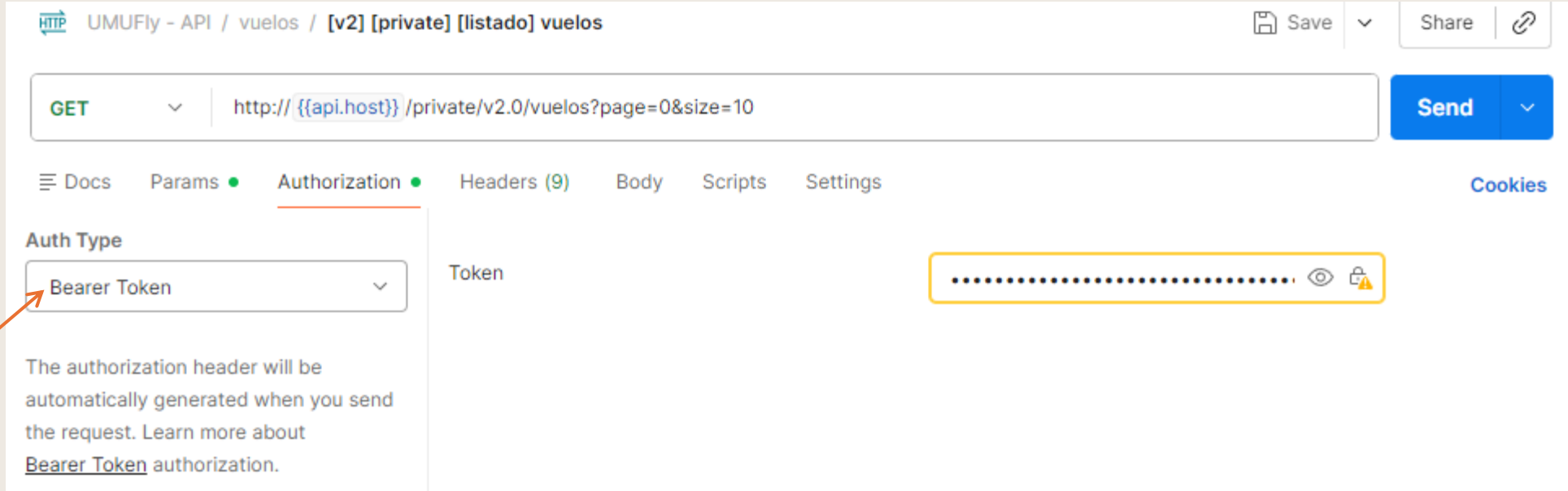
Spring Security

- Una vez generado el token, podremos usarlo como cabecera de autenticación
- Debermos configurar el método Bearer Token



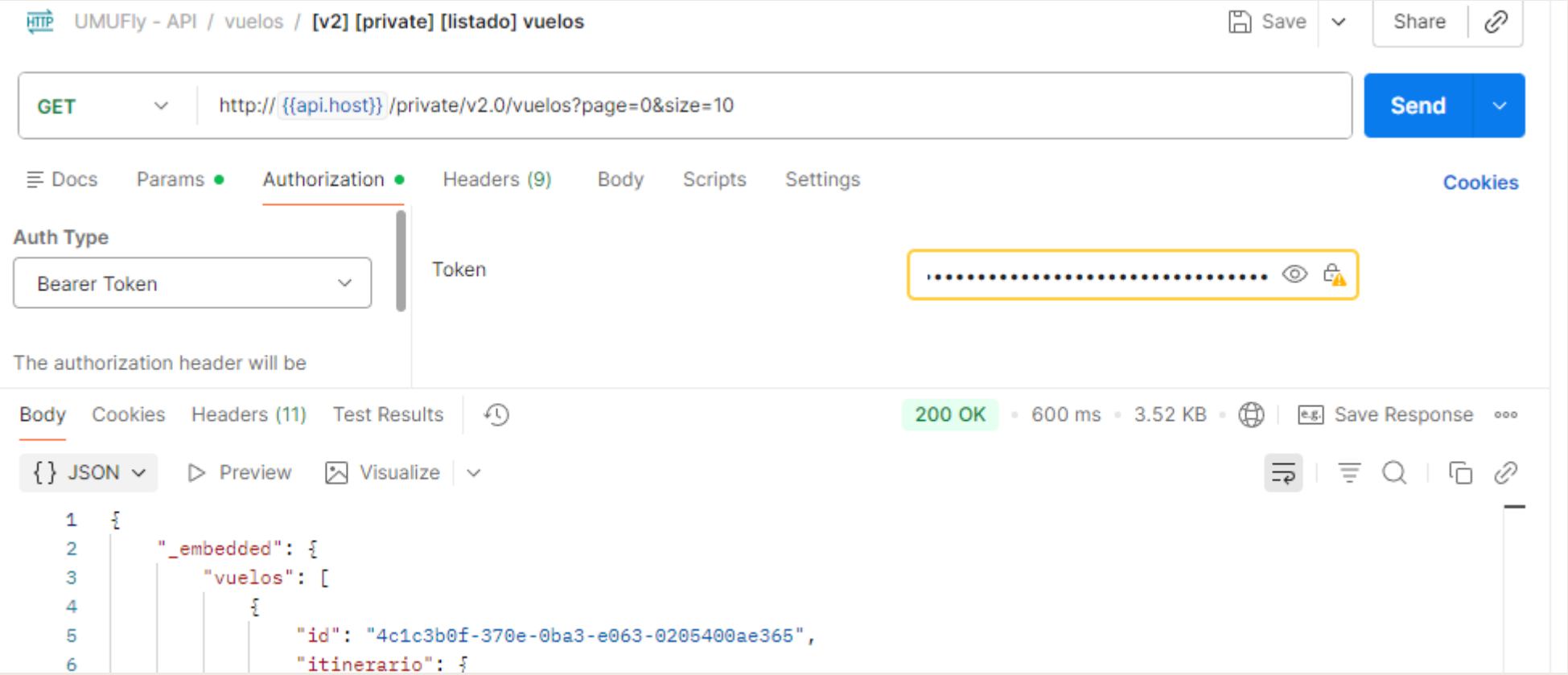
Spring Security

- Una vez generado el token, podremos usarlo como cabecera de autenticación
- Debermos configurar el método Bearer Token



Spring Security

- La llamada funcionará correctamente



Spring Security

- Ahora que tenemos autenticación, vamos a añadir también autorización
- Un usuario autenticado correctamente no tiene por qué acceder a todas las partes del sistema
- Gracias a los roles(Authorities), podemos restringir el acceso a determinados endpoints a usuarios con un rol o roles concretos

Spring Security

- A nuestro usuario de prueba, le pusimos por defecto ROLE_ADMIN
- Vamos a bloquear un endpoint para que sólo deje acceder a los usuarios con rol USR (ROLE_USR)

Spring Security

- El primer paso es activar la capa de autorización en nuestro SecurityConfig

```
@Configuration
//Autenticacion web
@EnableWebSecurity
//Autorizacion a nivel de metodo
@EnableMethodSecurity
public class SecurityConfig {

private final JwtAuthFilter jwtAuthFilter;

public SecurityConfig(JwtAuthFilter jwtAuthFilter) {
this.jwtAuthFilter = jwtAuthFilter;
}
…
…
```

Spring Security

- Añadimos en tu el endpoint que estamos probando la anotación @PreAuthorice
- Dentro de la misma pondremos una expresión booleana que dejará pasar o no la petición
- Inicialmente la ponemos con rol ADMIN para comprobar que funciona
- Posteriormente cambiamos a USER y dejará de funcionar

```
@GetMapping( Constants.PRIVATE_PREFIX + Constants.API_VERSION_2 + Constants.RESOURCE_VUELOS )
@PreAuthorize("hasRole('USER')")
public CollectionModel<VueloDTO> getVuelos( …… ) throws Exception {
   DocumentoIdentidad documento = authService.parseUserHeader( usuario );
  return pagedResourcesAssembler.toModel( listaVuelosQueryHandler.handle( ListaVuelosQuery.of( documento, page,
size ) ), vuelosModelAssemblerV2 );
 }
```

Spring Security

- También podemos observar que la cabecera creada UMU-Usuario ya no es necesaria
- Debemos cambiarla por AuthenticationPrincipal que cargará el usuario que viene en el token JWT

```
@GetMapping( Constants.PRIVATE_PREFIX + Constants.API_VERSION_2 + Constants.RESOURCE_VUELOS )
@PreAuthorize("hasRole('ADMIN')")
public CollectionModel<VueloDTO> getVuelos(@AuthenticationPrincipal UsuarioEntity usuario,
        @RequestParam( name = "page", defaultValue = "0" ) int page, @RequestParam( name = "size", defaultValue =
                        "25" ) int size ) throws Exception {
        log.info("Usuario {}",usuario.getEmail());
        DocumentoIdentidad documento = null;
        return pagedResourcesAssembler.toModel( listaVuelosQueryHandler.handle( ListaVuelosQuery.of( documento, page,
                size ) ), vuelosModelAssemblerV2 );
}
```

Spring Security

- PreAuthorize tiene un conjunto de método predeterminados
  - hasRole
  - hasAnyRole
  - isAuthenticated
  - Etc..
- Pero también puede llamar a cualquier método que tengamos en un componente
  - Para ello debemos ponerle nombre al componente @Component("nombre")
  - @PreAuthorize("@nombre.metodo")

Spring Security

- Qué parámetros puedo pasar en @PreAuthroize a mi método

Variable	Contenido
authentication	El objeto Authentication completo
principal	El usuario autenticado (tu entidad Usuario)
#paramName	Cualquier parámetro del método por nombre

```
@PreAuthorize("@authz.esPropioUsuario(authentication, #email)")
public ResponseEntity<?> actualizar(@PathVariable String email) { ... }
```

Tema 6. Documentación con OpenApi

Haciendo accesibles nuestras APIs

OpenAPI

- OpenAPI Specification (OAS) es un estándar para describir APIs RESTful de forma independiente de su implementación
- Por API RESTful entendemos una API REST
  - Orientada a recursos
  - Sin estado
  - Usa verbos HTTP: GET,POST,PUT,DELETE, etc.

OpenAPI

- Originalmente se conocía como Swagger Specification
- En 2015 se donó a Linux Foundation y pasó a llamare OpenAPI Specification
- Ventajas principales:
  - Define el contrato entre frontend y backend
  - Documentación integrada en el código fuente
  - Generación automática de clientes
  - Pruebas directas contra la API generada
  - Soporte para herramientas CI/CD

OpenAPI

- Integración con SprinBoot
- Podemos integrar OpenAPI en SpringBoot de dos maneras:
  - Generar solo el API
  - Generar el API e incluir interfaz gráfica para probarla
- Según la manera en la que queramos integrarla usaremos una u otra dependencia Maven

OpenAPI

- En este ejercicio vamos a usar la versión con interfaz grafica

Con interfaz gráfica

```
<dependency>
        <groupId>org.springdoc</groupId>
        <artifactId> springdoc-openapi-starter-webmvc-ui </artifactId>
        <version>3.0.2</version>
</dependency>
```

Sin interfaz gráfica

```
<dependency>
        <groupId>org.springdoc</groupId>
        <artifactId>springdoc-openapi-starter-webmvc-api</artifactId>
        <version>3.0.2</version>
</dependency>
```

OpenAPI

- En este ejercicio vamos a usar la versión con interfaz grafica
- Dentro de nuestro fichero application.properties incluiremos

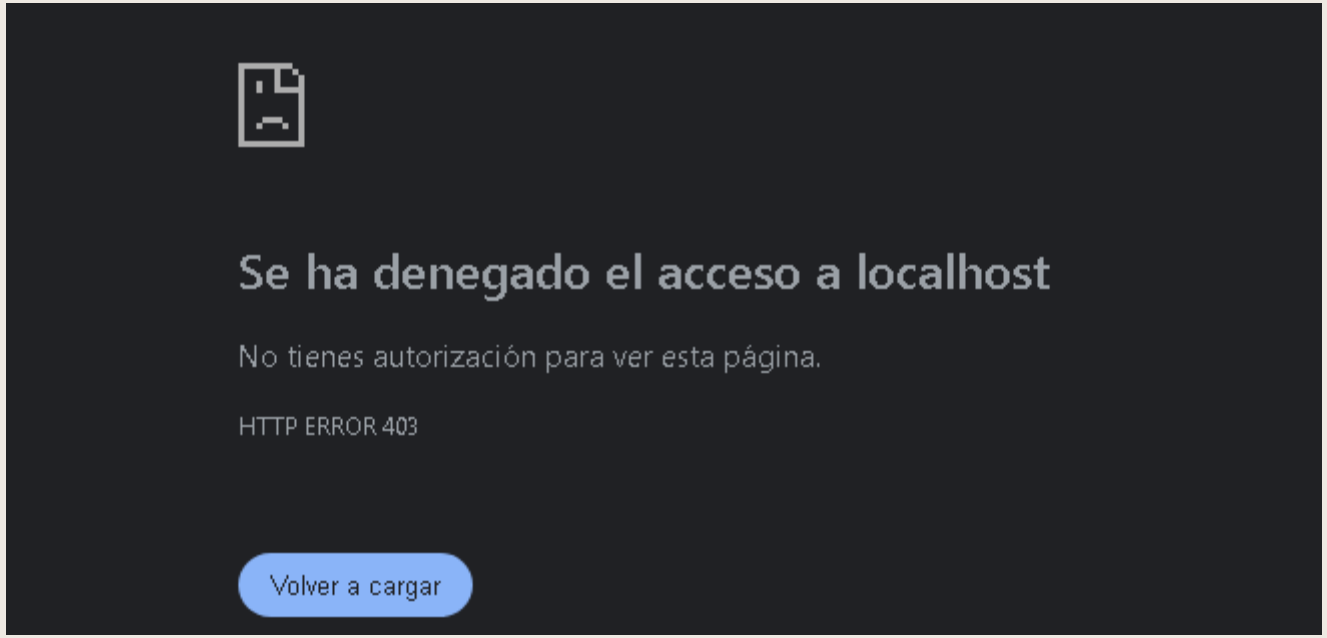
```
#Path donde se expone la documentación
springdoc.api-docs.path=/api-docs
#Index por defecto
springdoc.swagger-ui.path=/swagger-ui.html
#Interfaz grafica habilitada
springdoc.swagger-ui.enabled=true
```

OpenAPI

- Solo con la dependencia y las propiedades ya tengo levantada una UI donde ver información de nuestra API
- Si entro a http://localhost:8080/swagger-ui.html

OpenAPI

- Solo con la dependencia y las propiedades ya tengo levantada una UI donde ver información de nuestra API
- Si entro a http://localhost:8080/swagger-ui.html



OpenAPI

- ¿Qué ha pasado?
  - Nuestra aplicación tiene la configuración de seguridad habilitada
  - Todas las peticiones que no vayan a /auth requieren estar autenticadas
  - Debemos modificar el filtro para que deje entrar sin autenticación a la url donde expondremos nuestra API

OpenAPI

- ¿Qué ha pasado?
  - Nuestra aplicación tiene la configuración de seguridad habilitada

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
return http.csrf(csrf -> csrf.disable()).sessionManagement(session ->
        session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)).authorizeHttpRequests(
        auth -> auth.requestMatchers("/auth/login").permitAll()
        .anyRequest().authenticated()).addFilterBefore(jwtAuthFilter,
        UsernamePasswordAuthenticationFilter.class).build();
}
```

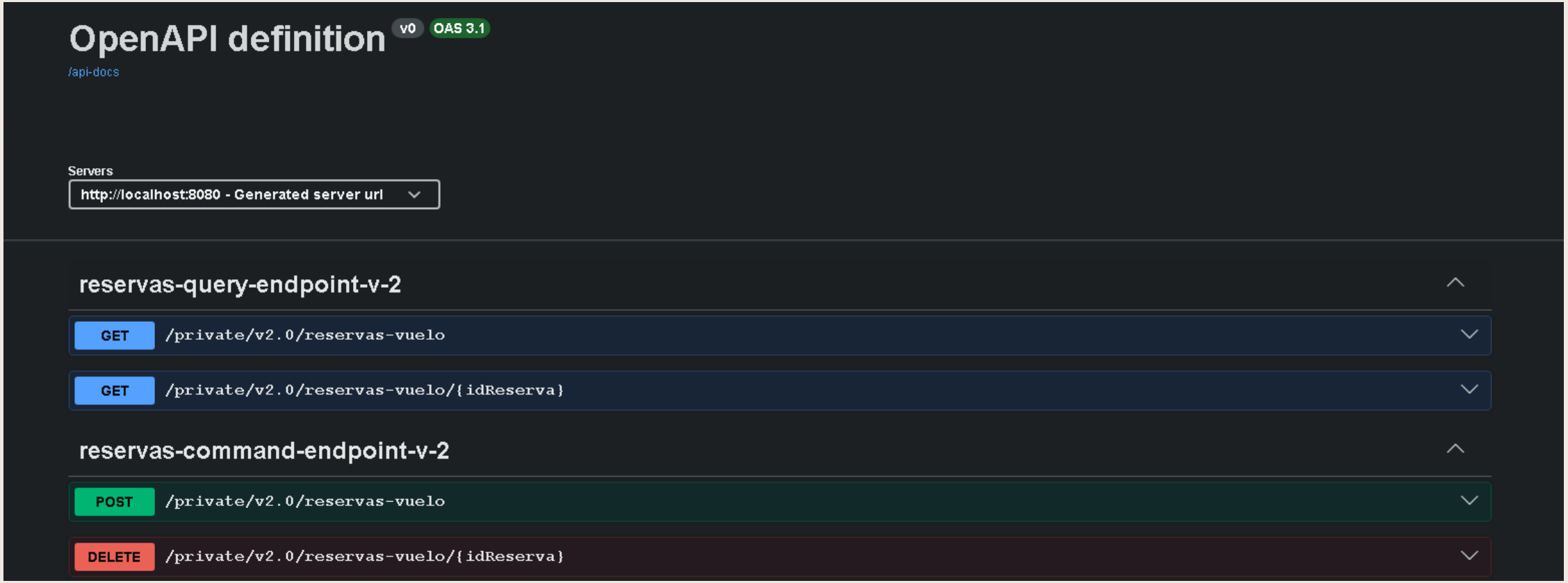
OpenAPI

- ¿Qué ha pasado?
  - Debemos incluir tanto la URL de la UI como la URL donde se expone la documentación

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
return http.csrf(csrf -> csrf.disable()).sessionManagement(session ->
         session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)).authorizeHttpRequests(
         auth -> auth.requestMatchers("/auth/login","/swagger-ui/**", "/api-docs/**").permitAll()
         .anyRequest().authenticated()).addFilterBefore(jwtAuthFilter,
         UsernamePasswordAuthenticationFilter.class).build();
}
```

OpenAPI

- Si ejecutamos ahora la misma consulta



OpenAPI

- Como nuestro código devuelve y consume datos conocidos (tipos primtivos o DTO) OpenAPI es capaz de pintar los datos

Code	Description
200	OK

Media type

/

Controls Accept header.

Example Value | Schema


```
{
  "_embedded": {
    "vuelos": [
      {
        "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
        "itinerario": {
          "salida": "2026-03-09T22:04:54.547Z",
          "llegada": "2026-03-09T22:04:54.547Z",
          "origen": "string",
          "destino": "string"
        },
        "tipo": "NACIONAL",
        "estado": "PENDIENTE",
```

OpenAPI

- Pero para gestionar y dar formato contamos con anotaciones que podemos añadir a nuestro código Java
- @OpenAPIDefinition
  - Define información global de la API.

```
@OpenAPIDefinition(
  info = @Info(
    title = "API de Vuelos",
    version = "1.0",
    description = "API para gestión de vuelos"
  )
)
```

OpenAPI

- Pero para gestionar y dar formato contamos con anotaciones que podemos añadir a nuestro código Java
- @OpenAPIDefinition
  - Define información global de la API.

```
@OpenAPIDefinition(
  info = @Info(
    title = "API de Vuelos",
    version = "1.0",
    description = "API para gestión de vuelos"
  )
)
```

OpenAPI

- @Tag
  - Sirve para agrupar endpoints en la documentación.

```
@RestController
@Tag(name = "Vuelosv2", description = "Operaciones sobre vuelos version2")
public class VuelosQueryEndpointV2


@Tag( name = "Parking", description = "Operaciones sobre parking" )
@RestController
public class ParkingQueryEndpoint {
```

OpenAPI

- @Tag

```
@RestController
@Tag(name = "Vuelosv2", description = "Operaciones
sobre vuelos version2")
public class VuelosQueryEndpointV2


@Tag( name = "Parking", description = "Operaciones
sobre parking" )
@RestController
public class ParkingQueryEndpoint {
```

Vuelosv2 Operaciones sobre vuelos version2

GET	/private/v2.0/reservas-vuelo
POST	/private/v2.0/reservas-vuelo
GET	/private/v2.0/vuelos
GET	/private/v2.0/vuelos/{idVuelo}
GET	/private/v2.0/reservas-vuelo/{idReserva}
DELETE	/private/v2.0/reservas-vuelo/{idReserva}

Vuelosv1 Operaciones sobre vuelos version1

GET	/private/v1.0/vuelos
GET	/private/v1.0/pruebas

Parking Operaciones sobre parking

GET	/private/v1.0/parkings
POST	/private/v1.0/parkings
GET	/private/v1.0/parkings/{idParking}
DELETE	/private/v1.0/parkings/{idParking}

OpenAPI

- @Operation
  - Describe un endpoint

```
@Operation( summary = "Obtener vuelos", description = "Devuelve la lista de vuelos paginada" )
@GetMapping( Constants.PRIVATE_PREFIX + Constants.API_VERSION_2 + Constants.RESOURCE_VUELOS )
public CollectionModel<VueloDTO> getVuelos( @AuthenticationPrincipal UsuarioEntity usuario …
```

OpenAPI

- @ApiResponses/ @ApiResponse
  - Lista de posibles respuestas de la API

```
@Operation( summary = "Obtener vuelos", description = "Devuelve la lista de vuelos paginada" )
@ApiResponses( {
        @ApiResponse( responseCode = "200", description = "OK" ),
        @ApiResponse( responseCode = "404", description = "No hay vuelos" )
} )
@GetMapping( Constants.PRIVATE_PREFIX + Constants.API_VERSION_2 + Constants.RESOURCE_VUELOS )
public CollectionModel<VueloDTO> getVuelos( @AuthenticationPrincipal UsuarioEntity usuario …
```

OpenAPI

- @Parameter
  - Permite describir un parámetro

```
@Operation( summary = "Obtener vuelos", description = "Devuelve la lista de vuelos paginada" )
@ApiResponses( {
        @ApiResponse( responseCode = "200", description = "OK" ),
        @ApiResponse( responseCode = "404", description = "No hay vuelos" )
} )
@GetMapping( Constants.PRIVATE_PREFIX + Constants.API_VERSION_2 + Constants.RESOURCE_VUELOS )
public CollectionModel<VueloDTO> getVuelos( @AuthenticationPrincipal UsuarioEntity usuario,
@Parameter(description = "Pagina inicio") @RequestParam( name = "page", defaultValue = "0" ) int page …
```

OpenAPI

- @Parameter
  - Permite describir un parámetro

```
@Operation( summary = "Obtener vuelos", description = "Devuelve la lista de vuelos paginada" )
@ApiResponses( {
        @ApiResponse( responseCode = "200", description = "OK" ),
        @ApiResponse( responseCode = "404", description = "No hay vuelos" )
} )
@GetMapping( Constants.PRIVATE_PREFIX + Constants.API_VERSION_2 + Constants.RESOURCE_VUELOS )
public CollectionModel<VueloDTO> getVuelos( @AuthenticationPrincipal UsuarioEntity usuario,
@Parameter(description = "Pagina inicio") @RequestParam( name = "page", defaultValue = "0" ) int page …
```

OpenAPI

- Existen más anotaciones, para ver la especificación podemos consultar

https://github.com/swagger-api/swagger-core/wiki/Swagger-2.X---Annotations

OpenAPI

- Ejercicio, documentar los endpoints de:
  - ReservasCommandEndpointV2
  - VuelosQueryEndpointV2

UNIVERSIDAD DE MURCIA

· 1272 ·